

AGL Lakehouse — Medallion Implementation Guide v2

Deeper build: full copy-paste-ready code, data quality guardrails, incremental Silver, and a business-first Gold layer

This is version 2 of the implementation guide, expanded on three fronts you asked for specifically: **(1)** every code block below is self-contained — imports included, no "continued from the cell above" assumptions — so you can copy any single snippet into a fresh cell and run it; **(2)** Silver now includes a reusable data-quality framework and a production-grade incremental (MERGE-based) pattern, not just full-rebuild examples; and **(3)** Gold is rebuilt around what each business function actually asks the data — Finance, Operations, Compliance, and Customer Success each get a named table and a stated question it answers, on top of the Data Scientist and ML Engineer tables from v1.

Still grounded in `AGL_Data_Dictionary_Verified.pdf`: no external database, all 31 tables land through Auto Loader from a Volume you populate by hand, and the two confirmed key mismatches (`vehicle_id` 4- vs 6-digit; `shipment_id` `SHIP_#####` vs `AGL-#####`) are still handled explicitly rather than glossed over.

Setting	Value used in this guide
Catalog	<code>workspace</code>
Landing schema (hosts the Volume + control tables)	<code>workspace.landing</code>
Volume	<code>/Volumes/workspace/landing/agl_volume/</code> , 6 subfolders mirroring your local folder names
Medallion schemas	<code>workspace.bronze</code> , <code>workspace.silver</code> , <code>workspace.gold</code>
Ingestion mechanism	Auto Loader (<code>cloudFiles</code>) for CSV/JSON/Parquet; manifest-table fallback for XML
Near-real-time detection	6 Databricks Jobs, one per domain folder, each on a File Arrival trigger
New in v2	Data-quality utility notebook, incremental MERGE pattern for Silver, 4 additional business-facing Gold tables, full unabridged CI/CD YAML for all 7 jobs, automated pytest suite in GitHub Actions
Notebooks built in this guide	<code>00_setup</code> , <code>00_data_quality_utils</code> , <code>01_bronze_ingestion</code> , <code>02_silver_transformation</code> , <code>02b_silver_incremental_merge</code> , <code>03_gold_layer</code>

PREREQUISITE

Unity Catalog-enabled workspace, a cluster with `com.databricks:spark-xml_2.12:0.18.0` installed (Part 1.2), and a GitHub account. If you built v1 of this guide already, this version's table and schema names are identical — running it just replaces the notebooks with deeper versions, nothing needs to be dropped first.

TABLE OF CONTENTS

Part 0 — Data Engineering Concepts You Need

Part 1 — Environment Setup: Catalog, Schemas, Volume, Cluster

Part 2 — Manual Upload Workflow

Part 3 — Bronze Layer: Auto Loader for All 31 Tables, All 4 Formats

Part 4 — Near-Real-Time Detection: 6 File-Arrival-Triggered Jobs

Part 5 — Silver Layer: Data Quality Framework + the 12 Industrial Transformations

Part 6 — Gold Layer: A Business-First Design

Part 7 — CI/CD: Databricks Asset Bundles + GitHub Actions (full YAML)

Part 8 — Troubleshooting Reference

Part 9 — Glossary

PART 0

Data Engineering Concepts You Need

0.1 The medallion architecture, and why three layers

CONCEPT

Bronze is raw data, landed as-is — its only job is to capture what arrived, reliably and exactly once, even if messy. **Silver** cleans, casts, deduplicates, joins and standardizes — "the truth, tidied up." **Gold** is shaped for a specific consumer — pre-aggregated, denormalized, named in business language. Keeping them separate means a transformation bug never costs you re-ingestion: Bronze never throws data away, so Silver/Gold can always be rebuilt from it.

0.2 Delta Lake essentials

Every table here is Delta by default. Four Delta features this guide leans on:

- **ACID transactions** — a failed write never leaves a half-written table.
- **MERGE INTO** — atomic upsert (update matched rows, insert new ones) — the backbone of Part 5's incremental pattern.
- **Time travel** — `SELECT * FROM t VERSION AS OF 3` or `TIMESTAMP AS OF '2026-07-01'` — a safety net if a bad Silver rebuild ships.
- **Schema enforcement/evolution** — writes fail on an unexpected column by default; `mergeSchema / overwriteSchema` opt into accepting the change (Part 5.13).

0.3 Auto Loader (`cloudFiles`)

CONCEPT

A Spark Structured Streaming source built to watch a cloud storage path (here, a Volume) and incrementally process only files it hasn't seen. State lives in a **checkpoint** — a hidden folder holding both the inferred schema and the list of already-processed files. Delete it to start over; keep it and re-running the same notebook is always safe, because already-processed files are simply skipped (this is what "idempotent" means in practice, not just in theory).

- **Schema inference & evolution** — first run samples files and infers a schema, saved to `cloudFiles.schemaLocation`. `cloudFiles.schemaEvolutionMode = "addNewColumns"` means a source adding a column later doesn't break the stream — the new run fails once with a controlled "schema changed" exception, and the next run picks up the new column.
- `trigger(availableNow=True)` processes everything currently in the folder, then stops — a "batch" run of a streaming query, correct for a scheduled/triggered Job. Omit it and the query polls forever (only sensible on an always-on cluster).
- **File arrival triggers** (Part 4) are a Job-level Databricks platform feature, not a notebook construct — Databricks watches a Volume path and starts a fresh Job run within roughly a minute of a new file appearing, with no cluster idling between uploads.

0.4 Partitioning, Z-Ordering and Liquid Clustering

CONCEPT

As Bronze/Silver tables grow past a few million rows, a query that only needs "last week's GPS pings" shouldn't scan the whole table. **Partitioning** (`partitionBy("event_date")`) physically separates files by date so a date filter skips whole

folders. **Z-Ordering** (`OPTIMIZE t ZORDER BY (vehicle_id)`) co-locates rows with the same key inside files, speeding up joins/filters on that key. **Liquid Clustering** (`CLUSTER BY (vehicle_id, event_date)`) is the newer Databricks-native alternative to both, requiring no manual re-partitioning as data grows. This guide's tables are small enough that none of this is required yet — Part 5.14 shows where to add it once they aren't.

0.5 Data quality as code, not vibes

"It looks right when I eyeball it" doesn't scale past a handful of tables. Part 5.1 builds three small, reusable Python functions — `assert_not_empty`, `null_rate`, `assert_max_null_rate` — that every Silver build calls before writing. A transformation that silently drops 90% of rows should throw an exception in your notebook, not quietly produce a half-empty Gold table three layers later.

0.6 Full rebuild vs. incremental MERGE

`CREATE OR REPLACE TABLE ... AS SELECT` ("full rebuild") is simple and correct at this dataset's current size (tens of thousands of rows) — every Silver table in Part 5.2–5.13 uses it. As tables grow, a full rebuild re-processes rows that haven't changed, wasting compute. Part 5.15 shows the alternative: a watermark (`updated_at`) plus `MERGE INTO`, processing only what changed since the last run — the same pattern the v1 beginner guide used for Neon sync, just applied here to a file-sourced Bronze table instead of a database.

0.7 The 12 industrial transformations, in one table

#	Transformation	One-line definition
1	Bucketing / Binning	Group a continuous numeric column into named ranges
2	Normalization / Standardization	Rescale a numeric column onto a common 0–1 or z-score range
3	Aggregation	Collapse many rows into a summary row via SUM/AVG/COUNT + GROUP BY
4	Joins and Merges	Combine two tables on a shared key
5	Pivoting / Unpivoting	Reshape rows into columns (wide) or columns into rows (long)
6	Deduplication	Keep exactly one row per logical key, deterministically
7	Filtering	Keep only rows that satisfy a business rule; route the rest to quarantine
8	Data Type Casting	Convert a column to the type it should have been
9	Window Functions	Compute a value relative to a partition/order without collapsing rows
10	Conditional Transformation	Derive a new column with CASE/WHEN business logic
11	Data Masking / Obfuscation	Irreversibly or reversibly hide PII while keeping the column usable
12	Schema Evolution	Accept new/changed source columns without breaking the pipeline

PART 1

Environment Setup: Catalog, Schemas, Volume, Cluster

1.1 Notebook: 00_setup

```
catalog = "workspace"

spark.sql(f"CREATE SCHEMA IF NOT EXISTS {catalog}.landing")
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {catalog}.bronze")
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {catalog}.silver")
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {catalog}.gold")

spark.sql(f"CREATE VOLUME IF NOT EXISTS {catalog}.landing.agl_volume")

domains = [
    "agl_data_output",
    "agl_fleet_data_output",
    "agl_driver_data_output",
    "agl_coldchain_data_output",
    "agl_delivery_data_output",
    "agl_external_data_output",
]
base = f"/Volumes/{catalog}/landing/agl_volume"
for d in domains:
    dbutils.fs.mkdirs(f"{base}/{d}")
    dbutils.fs.mkdirs(f"{base}/_checkpoints")
    dbutils.fs.mkdirs(f"{base}/_quarantine")

print("Schemas, volume and 6 domain folders ready.")
for d in domains:
    print(" -", f"{base}/{d}")
```

Expected result: `workspace.landing/bronze/silver/gold` all appear in Catalog Explorer; `/Volumes/workspace/landing/agl_volume/` shows 6 domain subfolders plus `_checkpoints` and `_quarantine`.

1.2 Cluster configuration

Create (or reuse) a cluster with these settings — write them down, you'll need the same runtime version referenced in Part 7's CI/CD YAML:

Setting	Value used in this guide
Databricks Runtime	15.4 LTS (or later LTS) — includes Delta Lake, Auto Loader, Unity Catalog support out of the box
Access mode	Single user or Shared, Unity Catalog enabled
Node type	Standard_DS3_v2 (or your cloud's small general-purpose equivalent) — this dataset does not need large nodes
Autoscaling workers	1–4

Install the XML reader library — 3 of the 31 sources (`vehicles`, `warehouses`, `proof_of_delivery`) are ingested as XML, and Spark has no built-in XML reader:

1. Compute → your cluster → **Libraries** tab → **Install New**.
2. Library source: **Maven**. Coordinates: `com.databricks:spark-xml_2.12:0.18.0`.

3. Install, then restart the cluster.

Expected result: `spark.read.format("xml")` no longer throws `ClassNotFoundException` when you reach Part 3.4.

PART 2

Manual Upload Workflow

You upload manually — nothing pulls data automatically from anywhere outside the Volume.

2.1 Option A — Catalog Explorer UI

1. Catalog → `workspace` → `landing` → Volumes → `agl_volume`.
2. Open the subfolder matching the domain (e.g. `agl_fleet_data_output`).
3. Click **Upload to this volume** and drop in the specific format file(s) listed in Part 3.2 — one format per table, not all 4.

2.2 Option B — Databricks CLI

```
databricks fs cp "agl_fleet_data_output/vehicles.xml"
/Volumes/workspace/landing/agl_volume/agl_fleet_data_output/
databricks fs cp "agl_fleet_data_output/trips.csv"
/Volumes/workspace/landing/agl_volume/agl_fleet_data_output/
databricks fs cp "agl_fleet_data_output/gps_tracking.parquet"
/Volumes/workspace/landing/agl_volume/agl_fleet_data_output/
databricks fs cp "agl_fleet_data_output/telematics.parquet"
/Volumes/workspace/landing/agl_volume/agl_fleet_data_output/
databricks fs cp "agl_fleet_data_output/maintenance_logs.csv"
/Volumes/workspace/landing/agl_volume/agl_fleet_data_output/
```

WATCH OUT

Do not use `--recursive` to copy an entire local folder — that uploads all 4 formats for every table, and Auto Loader (Part 3, globbing one extension per table) simply ignores the ones it isn't watching. Harmless, but wastes storage and confuses anyone browsing the volume later.

2.3 Naming convention for future drops

Upload new batches under a dated filename next to the original — e.g. `gps_tracking_20260801.parquet`. Every glob in Part 3 is `<table>*.<ext>`, so it's picked up automatically, and because it fires a File Arrival job (Part 4), nothing else needs to happen.

PART 3

Bronze Layer: Auto Loader for All 31 Tables, All 4 Formats

3.1 Design: one notebook, parameterized by domain

`01_bronze_ingestion` takes a `domain` widget (one of the 6 folder names) and only ingests that domain's sources — this is what lets Part 4 wire up 6 independent File Arrival jobs, each passing a different value to the same notebook.

```
dbutils.widgets.text("domain", "agl_fleet_data_output")
domain_param = dbutils.widgets.get("domain")

catalog = "workspace"
volume_path = "/Volumes/workspace/landing/agl_volume"
checkpoint_base = f"{volume_path}/_checkpoints"
bronze_schema = "bronze"

print(f"Running Bronze ingestion for domain: {domain_param}")
```

3.2 The full 31-source config table

domain (folder)	table	format	bronze target
agl_data_output	agl_shipment_data	CSV	shipment_data
	customers_master	CSV	dim_customer
	shippers_master	JSON	dim_shipper
	consignees_master	CSV	dim_consignee
agl_fleet_data_output	vehicles	XML	dim_vehicle
	trips	CSV	fleet_trips
	gps_tracking	Parquet	fleet_gps_tracking
	telematics	Parquet	fleet telematics
	maintenance_logs	CSV	fleet_maintenance_logs
agl_driver_data_output	drivers	CSV	dim_driver
	behavioral_events	JSON	driver_behavioral_events
	driver_logs	JSON	driver_logs
	app telematics	CSV	driver_app telematics
	fatigue_data	JSON	driver_fatigue_data
	pre_trip inspections	CSV	driver_pre_trip inspections
agl_coldchain_data_output	containers	CSV	dim_container
	warehouses	XML	dim_warehouse
	sensors	CSV	dim_sensor
	sensor_readings	Parquet	coldchain_sensor_readings
	reefer_status	Parquet	coldchain_reefer_status

	sensor_health	CSV	coldchain_sensor_health
agl_delivery_data_output	milestones	JSON	delivery_milestones
	exceptions	JSON	delivery_exceptions
	osd_logs	CSV	delivery_osd_logs
	proof_of_delivery	XML	delivery_proof_of_delivery
	delivery_metrics	CSV	delivery_metrics
agl_external_data_output	weather_data	JSON	ext_weather_data
	traffic_data	JSON	ext_traffic_data
	fuel_prices	CSV	ext_fuel_prices
	exchange_rates	CSV	ext_exchange_rates
	market_data	CSV	ext_market_data

```

bronze_sources = {
    "agl_data_output": [
        {"name": "agl_shipment_data", "format": "csv", "glob": "agl_shipment_data*.csv", "target":
"shipment_data"},
        {"name": "customers_master", "format": "csv", "glob": "customers_master*.csv", "target":
"dim_customer"},
        {"name": "shippers_master", "format": "json", "glob": "shippers_master*.json", "target":
"dim_shipper"},
        {"name": "consignees_master", "format": "csv", "glob": "consignees_master*.csv", "target":
"dim_consignee"},
    ],
    "agl_fleet_data_output": [
        {"name": "trips", "format": "csv", "glob": "trips*.csv", "target":
"fleet_trips"},
        {"name": "gps_tracking", "format": "parquet", "glob": "gps_tracking*.parquet", "target":
"fleet_gps_tracking"},
        {"name": "telematics", "format": "parquet", "glob": "telematics*.parquet", "target":
"fleet_telematics"},
        {"name": "maintenance_logs", "format": "csv", "glob": "maintenance_logs*.csv", "target":
"fleet_maintenance_logs"},
    ],
    "agl_driver_data_output": [
        {"name": "drivers", "format": "csv", "glob": "drivers*.csv", "target":
"dim_driver"},
        {"name": "behavioral_events", "format": "json", "glob": "behavioral_events*.json", "target":
"driver_behavioral_events"},
        {"name": "driver_logs", "format": "json", "glob": "driver_logs*.json", "target":
"driver_logs"},
        {"name": "app_telematics", "format": "csv", "glob": "app_telematics*.csv", "target":
"driver_app_telematics"},
        {"name": "fatigue_data", "format": "json", "glob": "fatigue_data*.json", "target":
"driver_fatigue_data"},
        {"name": "pre_trip_inspections", "format": "csv", "glob": "pre_trip_inspections*.csv", "target":
"driver_pre_trip_inspections"},
    ],
    "agl_coldchain_data_output": [
        {"name": "containers", "format": "csv", "glob": "containers*.csv", "target":
"dim_container"},
        {"name": "sensors", "format": "csv", "glob": "sensors*.csv", "target":
"dim_sensor"},
        {"name": "sensor_readings", "format": "parquet", "glob": "sensor_readings*.parquet", "target":
"coldchain_sensor_readings"},
        {"name": "reefer_status", "format": "parquet", "glob": "reefer_status*.parquet", "target":
"coldchain_reefer_status"},
        {"name": "sensor_health", "format": "csv", "glob": "sensor_health*.csv", "target":
"coldchain_sensor_health"},
    ]
}

```

```

    ],
    "agl_delivery_data_output": [
        {"name": "milestones", "format": "json", "glob": "milestones*.json", "target":
"delivery_milestones"},
        {"name": "exceptions", "format": "json", "glob": "exceptions*.json", "target":
"delivery_exceptions"},
        {"name": "osd_logs", "format": "csv", "glob": "osd_logs*.csv", "target":
"delivery_osd_logs"},
        {"name": "delivery_metrics", "format": "csv", "glob": "delivery_metrics*.csv", "target":
"delivery_metrics"},
    ],
    "agl_external_data_output": [
        {"name": "weather_data", "format": "json", "glob": "weather_data*.json", "target":
"ext_weather_data"},
        {"name": "traffic_data", "format": "json", "glob": "traffic_data*.json", "target":
"ext_traffic_data"},
        {"name": "fuel_prices", "format": "csv", "glob": "fuel_prices*.csv", "target":
"ext_fuel_prices"},
        {"name": "exchange_rates", "format": "csv", "glob": "exchange_rates*.csv", "target":
"ext_exchange_rates"},
        {"name": "market_data", "format": "csv", "glob": "market_data*.csv", "target":
"ext_market_data"},
    ],
}
print(f"{sum(len(v) for v in bronze_sources.values())} CSV/JSON/Parquet sources configured across 6
domains.")

```

3.3 Ingestion log table (new in v2) — one row per run, per source

```

from pyspark.sql import Row
from datetime import datetime

spark.sql(f"""
CREATE TABLE IF NOT EXISTS {catalog}.landing._ingestion_log (
    target_table STRING,
    domain STRING,
    format STRING,
    status STRING,
    error_message STRING,
    run_timestamp TIMESTAMP
)""")

def log_ingestion(target_table, domain, fmt, status, error_message=None):
    spark.createDataFrame([Row(
        target_table=target_table, domain=domain, format=fmt,
        status=status, error_message=error_message, run_timestamp=datetime.utcnow()
    )]).write.mode("append").saveAsTable(f"{catalog}.landing._ingestion_log")

print("Ingestion log table ready.")

```

3.4 Generic ingestion function — CSV, JSON and Parquet

```

def ingest_source(domain_folder, src):
    reader = (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", src["format"])
        .option("cloudFiles.schemaLocation", f"{checkpoint_base}/{src['target']}_schema")
        .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    )
    if src["format"] == "csv":
        reader = reader.option("header", "true")
    if src["format"] == "json":
        reader = reader.option("multiline", "true")

```

```

df = reader.load(f"{volume_path}/{domain_folder}/{src['glob']}")

(df.writeStream
 .format("delta")
 .option("checkpointLocation", f"{checkpoint_base}/{src['target']}_ckpt")
 .option("mergeSchema", "true")
 .outputMode("append")
 .trigger(availableNow=True)
 .toTable(f"{catalog}.{bronze_schema}.{src['target']}")
 .awaitTermination())

results = []
for src in bronze_sources.get(domain_param, []):
    try:
        ingest_source(domain_param, src)
        log_ingestion(src["target"], domain_param, src["format"], "OK")
        results.append((src["target"], "OK"))
        print(f"{src['target']}: batch complete.")
    except Exception as e:
        log_ingestion(src["target"], domain_param, src["format"], "FAILED", str(e))
        results.append((src["target"], f"FAILED: {e}"))
        print(f"{src['target']}: FAILED - {e}")

ok_count = sum(1 for r in results if r[1] == "OK")
print(f"\n{domain_param}: {ok_count}/{len(results)} sources ingested.")

```

Expected result: when `domain = agl_fleet_data_output`, 4 "batch complete" lines print (trips, gps_tracking, telematics, maintenance_logs — vehicles is XML, handled next) and a new row appears in `workspace.landing._ingestion_Log` per source.

3.5 The XML path — generalized manifest-table fallback

```

from pyspark.sql.functions import col
from pyspark.sql import Row
from datetime import datetime

spark.sql(f"""
CREATE TABLE IF NOT EXISTS {catalog}.landing._processed_xml_files (
    file_path STRING, target_table STRING, processed_at TIMESTAMP
)""")

def ingest_xml_source(domain_folder, glob_prefix, row_tag, target_table, column_map):
    all_files = [f.path for f in dbutils.fs.ls(f"{volume_path}/{domain_folder}")
                  if f.name.startswith(glob_prefix) and f.name.endswith(".xml")]
    already = set(r["file_path"] for r in spark.sql(f"""
        SELECT file_path FROM {catalog}.landing._processed_xml_files WHERE target_table = '{target_table}'
    """).collect())
    new_files = [f for f in all_files if f not in already]

    if not new_files:
        print(f"{target_table}: no new XML files.")
        log_ingestion(target_table, domain_folder, "xml", "OK", "no new files")
        return

    try:
        df_raw = spark.read.format("xml").option("rowTag", row_tag).load(new_files)
        df_clean = df_raw.select(*[col(src).alias(dst) for src, dst in column_map.items()])
        df_clean.write.mode("append").option("mergeSchema", "true").saveAsTable(f"{catalog}.{bronze_schema}.
{target_table}")

        spark.createDataFrame(
            [Row(file_path=f, target_table=target_table, processed_at=datetime.utcnow()) for f in new_files]
        ).write.mode("append").saveAsTable(f"{catalog}.landing._processed_xml_files")
        log_ingestion(target_table, domain_folder, "xml", "OK")
        print(f"{target_table}: processed {len(new_files)} new XML file(s).")
    
```

```

except Exception as e:
    log_ingestion(target_table, domain_folder, "xml", "FAILED", str(e))
    print(f"{target_table}: FAILED - {e}")

# --- vehicles.xml : root , row tag ---
vehicle_column_map = {
    "vehicleid": "vehicle_id", "licenseplate": "license_plate", "model": "model", "year": "year",
    "fuelcapacityl": "fuel_capacity_l", "avgconsumptionlper100km": "avg_consumption_l_per_100km",
    "currentfuellevel": "current_fuel_level", "odometerreadingkm": "odometer_reading_km",
    "trucktype": "truck_type", "maxloadtonnes": "max_load_tonnes", "status": "status",
    "assigneddriver": "assigned_driver", "assignedcorridor": "assigned_corridor",
    "insuranceexpiry": "insurance_expiry", "lastinspectiondate": "last_inspection_date",
    "nextinspectiondate": "next_inspection_date",
}
if domain_param == "agl_fleet_data_output":
    ingest_xml_source("agl_fleet_data_output", "vehicles", "Vehicle", "dim_vehicle", vehicle_column_map)

# --- warehouses.xml : root , row tag ---
warehouse_column_map = {
    "warehouseid": "warehouse_id", "warehousename": "warehouse_name", "address": "address",
    "totalaream2": "total_area_m2", "coolingcapacitykw": "cooling_capacity_kw",
    "numberofbays": "number_of_bays", "producttypes": "product_types", "temperaturezone": "temperature_zone",
    "powerbackuphours": "power_backup_hours", "generatorcapacitykva": "generator_capacity_kva",
    "managername": "manager_name", "contactphone": "contact_phone", "contactemail": "contact_email",
    "operatinghours": "operating_hours", "certification": "certification", "status": "status",
}
if domain_param == "agl_coldchain_data_output":
    ingest_xml_source("agl_coldchain_data_output", "warehouses", "Warehouse", "dim_warehouse",
warehouse_column_map)

# --- proof_of_delivery.xml : root , row tag ---
pod_column_map = {
    "podid": "pod_id", "shipmentid": "shipment_id", "recipientname": "recipient_name",
    "recipientcontact": "recipient_contact", "recipientemail": "recipient_email",
    "recipientsignature": "recipient_signature", "deliveryagentname": "delivery_agent_name",
    "deliveryagentsignature": "delivery_agent_signature", "deliverytimestamp": "delivery_timestamp",
    "deliverylocation": "delivery_location", "cargoconditionsummary": "cargo_condition_summary",
    "cargoconditiondetails": "cargo_condition_details", "photoevidence": "photo_evidence",
    "temperatureatdeliveryc": "temperature_at_delivery_c", "humidityatdeliverypercent":
"humidity_at_delivery_percent",
    "deliverynotes": "delivery_notes", "receivedby": "received_by",
}
if domain_param == "agl_delivery_data_output":
    ingest_xml_source("agl_delivery_data_output", "proof_of_delivery", "POD", "delivery_proof_of_delivery",
pod_column_map)

```

WATCH OUT

The row tag for `proof_of_delivery.xml` is `<POD>`, not `<ProofOfDelivery>` — root element and row element have different names. Always open the actual file and check both before wiring up `rowTag`.

NOTE

`warehouses.xml` has a real encoding defect: `temperature_zone` contains "Ambient (15Â°C to 25Â°C)" — a mojibake double-encoding of °. Bronze lands it as-is; Part 5.3 fixes it.

3.6 Verify Bronze after each domain

```

domains_to_check = ["agl_data_output", "agl_fleet_data_output", "agl_driver_data_output",
                    "agl_coldchain_data_output", "agl_delivery_data_output", "agl_external_data_output"]

log_df = spark.table("workspace.landing.ingestion_log")
summary = (log_df.groupBy("domain", "target_table")
            .agg({"run_timestamp": "max"})
            .withColumnRenamed("max(run_timestamp)", "last_run"))

```

```
summary.orderBy("domain", "target_table").show(50, truncate=False)

failed = log_df.filter("status = 'FAILED'")
if failed.count() > 0:
    print("FAILURES DETECTED:")
    failed.select("target_table", "domain", "error_message", "run_timestamp").show(20, truncate=False)
else:
    print("No ingestion failures logged.")
```

Expected result: one row per (domain, target_table), and an empty failure list once all 6 domains have run at least once.

PART 4

Near-Real-Time Detection: 6 File-Arrival-Triggered Jobs

4.1 Why 6 jobs and not 1

A Databricks Job can only have **one** trigger. To get "upload a file into any of the 6 folders → that folder's data gets processed automatically," the shape is **one Job per folder**, each watching its own path, each running `01_bronze_ingestion` with a different `domain` parameter.

4.2 Create the 6 jobs (manual UI steps)

1. Workflows → **Create Job**. Name it e.g. `AGL_Bronze_Fleet`.
2. Task: name `ingest`, type Notebook, path `01_bronze_ingestion`.
3. Task **Parameters**: add base parameter `domain` = `agl_fleet_data_output` (match the folder name exactly).
4. Compute: a small Job Cluster.
5. **Schedules & Triggers** → Add trigger → Trigger type **File arrival** → path `/Volumes/workspace/landing/agl_volume/agl_fleet_data_output/`.
6. **Notifications** → Add notification → check **On failure** → enter your email.
7. Save.

Job name	domain parameter / watched path
AGL_Bronze_Shipment	agl_data_output
AGL_Bronze_Fleet	agl_fleet_data_output
AGL_Bronze_Driver	agl_driver_data_output
AGL_Bronze_Coldchain	agl_coldchain_data_output
AGL_Bronze_Delivery	agl_delivery_data_output
AGL_Bronze_External	agl_external_data_output

Expected result: upload a new file into `agl_coldchain_data_output/` — within about a minute, `AGL_Bronze_Coldchain` starts automatically, ingests just that domain, and stops.

4.3 Monitoring all 6 jobs at once

```
%sql
SELECT job_name, run_id, run_page_url, result_state, start_time, end_time
FROM system.lakeflow.job_run_timeline
WHERE job_name LIKE 'AGL_Bronze_%'
ORDER BY start_time DESC
LIMIT 50
```

Uses Unity Catalog's system tables (`system.lakeflow.job_run_timeline`) to see every Bronze job's recent runs in one query instead of clicking through 6 job pages. Table name may differ slightly by workspace region/version — if it errors, check Workflows → Job runs and filter by name prefix `AGL_Bronze_` instead.

PART 5

Silver Layer: Data Quality Framework + the 12 Industrial Transformations

Two notebooks: `00_data_quality_utils` (shared helpers, included via `%run`) and `02_silver_transformation` (the 12 transformations, chained into full table builds). Every code block below is self-contained.

5.1 Notebook: `00_data_quality_utils` — the reusable guardrails

```
from pyspark.sql import DataFrame
from pyspark.sql.functions import col

def assert_not_empty(df: DataFrame, table_name: str) -> int:
    """Raise if a transformation produced zero rows - stops a silent empty-table bug from reaching Gold."""
    n = df.count()
    if n == 0:
        raise ValueError(f"{table_name}: zero rows after transformation - aborting.")
    return n

def null_rate(df: DataFrame, column: str) -> float:
    total = df.count()
    if total == 0:
        return 0.0
    nulls = df.filter(col(column).isNull()).count()
    return nulls / total

def assert_max_null_rate(df: DataFrame, column: str, max_rate: float, table_name: str) -> float:
    """Raise if a column's null rate exceeds an expected threshold - catches a broken join/cast early."""
    rate = null_rate(df, column)
    if rate > max_rate:
        raise ValueError(
            f"{table_name}.{column}: null rate {rate:.1%} exceeds threshold {max_rate:.1%} - "
            f"check upstream join keys or casting logic before trusting this table."
        )
    return rate

def row_count_delta(before_df: DataFrame, after_df: DataFrame, table_name: str, max_drop_pct: float = 0.5):
    """Raise if a filter/dedup step dropped more rows than expected - a >50% drop is almost always a bug."""
    before_n, after_n = before_df.count(), after_df.count()
    if before_n == 0:
        return 0.0
    drop_pct = 1 - (after_n / before_n)
    if drop_pct > max_drop_pct:
        raise ValueError(
            f"{table_name}: row count dropped {drop_pct:.1%} ({before_n} -> {after_n}), "
            f"exceeding the {max_drop_pct:.1%} expected threshold."
        )
    print(f"{table_name}: {before_n} -> {after_n} rows ({drop_pct:.1%} change)")
    return drop_pct

print("Data quality utilities loaded: assert_not_empty, null_rate, assert_max_null_rate, row_count_delta")
```

Include this at the top of every Silver/Gold notebook with `%run ./00_data_quality_utils` (its own cell, must be first). All four functions are called throughout the rest of Part 5.

5.2 Data Type Casting

```
from pyspark.sql.functions import regexp_replace, from_json, col
from pyspark.sql.types import StructType, StructField, DoubleType, ArrayType, StringType

# 1. fix the warehouses.xml mojibake (Part 3.5 note)
warehouse_fixed = (
    spark.table("workspace.bronze.dim_warehouse")
    .withColumn("temperature_zone", regexp_replace(col("temperature_zone"), "Â°", "°"))
)
assert_not_empty(warehouse_fixed, "dim_warehouse")
warehouse_fixed.write.mode("overwrite").option("overwriteSchema",
"true").saveAsTable("workspace.silver.dim_warehouse")

# 2. app telematics.last_known_location: Python-dict-repr string -> real lat/lon doubles
loc_schema = StructType([StructField("lat", DoubleType()), StructField("lon", DoubleType())])
app_telematics_fixed = (
    spark.table("workspace.bronze.driver_app_telematics")
    .withColumn("location_json", regexp_replace(col("last_known_location"), "'", "'"))
    .withColumn("location_parsed", from_json(col("location_json"), loc_schema))
    .withColumn("last_known_lat", col("location_parsed.lat"))
    .withColumn("last_known_lon", col("location_parsed.lon"))
    .drop("last_known_location", "location_json", "location_parsed")
)
assert_not_empty(app_telematics_fixed, "driver_app_telematics")
assert_max_null_rate(app_telematics_fixed, "last_known_lat", 0.05, "driver_app_telematics")
app_telematics_fixed.write.mode("overwrite").option("overwriteSchema",
"true").saveAsTable("workspace.silver.driver_app_telematics")

# 3. pre_trip inspections.inspection_items: embedded JSON array string -> parsed array
items_schema = ArrayType(StructType([
    StructField("item", StringType()), StructField("status", StringType()), StructField("notes",
StringType())
]))
pti_fixed = (
    spark.table("workspace.bronze.driver_pre_trip_inspections")
    .withColumn("inspection_items_parsed", from_json(col("inspection_items"), items_schema))
)
assert_not_empty(pti_fixed, "driver_pre_trip_inspections")
pti_fixed.write.mode("overwrite").option("overwriteSchema",
"true").saveAsTable("workspace.silver.driver_pre_trip_inspections")

print("Type casting complete: dim_warehouse, driver_app_telematics, driver_pre_trip_inspections")
```

Expected result: `silver.driver_app_telematics` has real double columns `Last_known_Lat / Last_known_Lon`; the null-rate assertion passes (nearly every row should parse); `silver.dim_warehouse.temperature_zone` reads "15°C to 25°C" correctly.

5.3 Deduplication (industrial pattern: keep-latest-by-window, not blind dropDuplicates)

```
from pyspark.sql.functions import col, desc, row_number
from pyspark.sql.window import Window

w = Window.partitionBy("shipment_id").orderBy(desc("updated_at"))
shipment_raw = spark.table("workspace.bronze.shipment_data")
shipment_dedup = (
    shipment_raw
    .withColumn("rn", row_number().over(w))
    .filter("rn = 1")
    .drop("rn")
)
row_count_delta(shipment_raw, shipment_dedup, "shipment_data (dedup)", max_drop_pct=0.05)
assert_not_empty(shipment_dedup, "shipment_data")
```



```
shipment_dedup.write.mode("overwrite").option("overwriteSchema",
"true").saveAsTable("workspace.silver.shipment_data")
```

`row_count_delta` is set to allow only a 5% drop here — dedup on a table that's supposed to already have unique `shipment_id` values (per the Data Dictionary) should remove almost nothing; a bigger drop would mean the source actually has real duplicates worth investigating, not silently discarding.

5.4 Filtering

```
from pyspark.sql.functions import lit

shipment_silver = spark.table("workspace.silver.shipment_data")
bad_rows = shipment_silver.filter("weight_kg <= 0 OR volume_m3 <= 0 OR piece_count <= 0")
good_rows = shipment_silver.filter("weight_kg > 0 AND volume_m3 > 0 AND piece_count > 0")

bad_rows.withColumn("quarantine_reason", lit("non-positive weight/volume/piece_count")) \
    .write.mode("append").option("mergeSchema",
"true").saveAsTable("workspace.silver.shipment_data_quarantine")

row_count_delta(shipment_silver, good_rows, "shipment_data (filter)", max_drop_pct=0.10)
good_rows.write.mode("overwrite").option("overwriteSchema",
"true").saveAsTable("workspace.silver.shipment_data")
print(f"Quarantined {bad_rows.count()} row(s) into workspace.silver.shipment_data_quarantine")
```

5.5 Conditional Transformation

```
spark.sql("""
CREATE OR REPLACE TABLE workspace.silver.dim_customer AS
SELECT *,
    CASE
        WHEN credit_limit >= 75000 THEN 'High Value'
        WHEN credit_limit >= 30000 THEN 'Standard'
        ELSE 'Low Value'
    END AS customer_value_tier
FROM workspace.bronze.dim_customer
""")

spark.sql("""
CREATE OR REPLACE TABLE workspace.silver.shipment_data AS
SELECT *,
    CASE
        WHEN actual_delivery_date IS NULL AND delivery_deadline < current_timestamp() THEN 'Overdue - In Transit'
        WHEN actual_delivery_date > delivery_deadline THEN 'Delivered Late'
        WHEN actual_delivery_date IS NOT NULL THEN 'Delivered On Time'
        ELSE 'In Transit - On Schedule'
    END AS delivery_status_flag,
    ROUND(freight_charges * penalty_rate, 2) AS penalty_exposure_amount
FROM workspace.silver.shipment_data
""")

print("Conditional flags added: dim_customer.customer_value_tier,
shipment_data.delivery_status_flag/penalty_exposure_amount")
```

FEEDS GOLD

`customer_value_tier` and `delivery_status_flag` are what Part 6.2/6.3's Finance and Customer Success tables group by — this is the transformation that makes "how are our High Value customers' shipments performing" answerable at all.

5.6 Bucketing / Binning

```
spark.sql("""
CREATE OR REPLACE TABLE workspace.silver.dim_driver AS
SELECT *,
    CASE
        WHEN total_experience_years < 2 THEN 'Junior (0-2y)'
        WHEN total_experience_years < 5 THEN 'Mid (2-5y)'
        WHEN total_experience_years < 10 THEN 'Senior (5-10y)'
        ELSE 'Veteran (10y+)'
    END AS experience_band
FROM workspace.bronze.dim_driver
""")

spark.sql("""
CREATE OR REPLACE TABLE workspace.silver.dim_vehicle AS
SELECT *,
    (year(current_date()) - CAST(year AS INT)) AS vehicle_age_years,
    CASE
        WHEN (year(current_date()) - CAST(year AS INT)) <= 3 THEN 'New (0-3y)'
        WHEN (year(current_date()) - CAST(year AS INT)) <= 7 THEN 'Mid-life (4-7y)'
        ELSE 'Aging (8y+)'
    END AS vehicle_age_band
FROM workspace.bronze.dim_vehicle
""")

spark.sql("""
CREATE OR REPLACE TABLE workspace.silver.shipment_data AS
SELECT *,
    CASE
        WHEN weight_kg < 1000 THEN 'Light (<1t)'
        WHEN weight_kg < 5000 THEN 'Medium (1-5t)'
        WHEN weight_kg < 20000 THEN 'Heavy (5-20t)'
        ELSE 'Bulk (20t+)'
    END AS weight_class
FROM workspace.silver.shipment_data
""")

print("Buckets added: dim_driver.experience_band, dim_vehicle.vehicle_age_band, shipment_data.weight_class")
```

5.7 Normalization / Standardization

```
from pyspark.sql.functions import mean as fmean, stddev, min as fmin, max as fmax, col

behavioral_events = spark.table("workspace.silver.driver_behavioral_events") \
    if spark.catalog.tableExists("workspace.silver.driver_behavioral_events") \
    else spark.table("workspace.bronze.driver_behavioral_events")

stats = behavioral_events.agg(
    fmean("speed_kmh").alias("mu"), stddev("speed_kmh").alias("sigma"),
    fmin("speed_kmh").alias("lo"), fmax("speed_kmh").alias("hi")
).collect()[0]

behavioral_scaled = (
    behavioral_events
    .withColumn("speed_kmh_zscore", (col("speed_kmh") - stats["mu"]) / stats["sigma"])
    .withColumn("speed_kmh_minmax", (col("speed_kmh") - stats["lo"]) / (stats["hi"] - stats["lo"]))
)

assert_not_empty(behavioral_scaled, "driver_behavioral_events")
behavioral_scaled.write.mode("overwrite").option("overwriteSchema",
"true").saveAsTable("workspace.silver.driver_behavioral_events")
print(f"Normalized speed_kmh: mu={stats['mu']:.2f}, sigma={stats['sigma']:.2f}, range=[{stats['lo']:.2f},
{stats['hi']:.2f}]")
```

Z-score flags outlier speeds for anomaly detection; min-max gives ML models a bounded 0–1 feature — both derived from the same source column for two different downstream consumers.

5.8 Window Functions

```
from pyspark.sql.functions import rank, avg, desc, row_number, col
from pyspark.sql.window import Window

# 1. rank drivers by rating within their home terminal
w_rank = Window.partitionBy("home_terminal").orderBy(desc("rating"))
driver_ranked = spark.table("workspace.silver.dim_driver").withColumn("rating_rank_in_terminal",
rank().over(w_rank))
driver_ranked.write.mode("overwrite").option("overwriteSchema",
"true").saveAsTable("workspace.silver.dim_driver")

# 2. latest sensor reading per sensor (row_number, not dedup - full history stays in bronze)
w_latest = Window.partitionBy("sensor_id").orderBy(desc("timestamp"))
latest_readings = (
    spark.table("workspace.bronze.coldchain_sensor_readings")
    .withColumn("rn", row_number().over(w_latest))
    .filter("rn = 1").drop("rn")
)
latest_readings.write.mode("overwrite").option("overwriteSchema",
"true").saveAsTable("workspace.silver.coldchain_latest_reading_per_sensor")

# 3. 7-day rolling average temperature per corridor
w_roll = Window.partitionBy("corridor_name").orderBy("event_date").rowsBetween(-6, 0)
weather_with_date = spark.table("workspace.bronze.ext_weather_data").withColumn("event_date",
col("timestamp").cast("date"))
weather_rolling = weather_with_date.withColumn("temp_c_7day_avg", avg("temperature_c").over(w_roll))
weather_rolling.write.mode("overwrite").option("overwriteSchema",
"true").saveAsTable("workspace.silver.ext_weather_data")

print("Window functions applied: dim_driver ranking, latest-per-sensor, 7-day rolling weather average")
```

5.9 Aggregation

```
spark.sql("""
CREATE OR REPLACE TABLE workspace.silver.fleet_maintenance_cost_by_vehicle AS
SELECT vehicle_id,
    COUNT(*) AS service_count,
    SUM(cost) AS total_maintenance_cost,
    AVG(cost) AS avg_maintenance_cost,
    MAX(cost) AS max_single_cost,
    SUM(CASE WHEN breakdown_recorded THEN 1 ELSE 0 END) AS breakdown_count
FROM workspace.bronze.fleet_maintenance_logs
GROUP BY vehicle_id
""")

spark.sql("""
CREATE OR REPLACE TABLE workspace.silver.ext_corridor_conditions_daily AS
SELECT corridor_name, CAST(timestamp AS DATE) AS event_date,
    AVG(temperature_c) AS avg_temp_c, MAX(precipitation_mm) AS max_precip_mm,
    AVG(humidity_percent) AS avg_humidity_pct
FROM workspace.bronze.ext_weather_data
GROUP BY corridor_name, CAST(timestamp AS DATE)
""")

print("Aggregates built: fleet_maintenance_cost_by_vehicle, ext_corridor_conditions_daily")
```

5.10 Joins and Merges (clean joins, then the mismatched-key pattern)

```
spark.sql("""
CREATE OR REPLACE TABLE workspace.silver.fleet_trip_enriched AS
SELECT t.*, v.truck_type, v.max_load_tonnes, v.vehicle_age_band, v.status AS vehicle_status
FROM workspace.silver.fleet_trips t
LEFT JOIN workspace.silver.dim_vehicle v ON t.vehicle_id = v.vehicle_id
""")

spark.sql("""
CREATE OR REPLACE TABLE workspace.silver.coldchain_reading_enriched AS
SELECT r.*, c.product_type, c.setpoint_temp_c, c.temp_min_c, c.temp_max_c,
       CASE WHEN r.temperature_c < c.temp_min_c OR r.temperature_c > c.temp_max_c THEN 1 ELSE 0 END AS
temp_excursion_flag
FROM workspace.bronze.coldchain_sensor_readings r
LEFT JOIN workspace.silver.dim_container c ON r.container_id = c.container_id
""")

# validate the enriched joins actually matched (per Data Dictionary Part 8, these two ARE confirmed clean)
fte = spark.table("workspace.silver.fleet_trip_enriched")
assert_max_null_rate(fte, "truck_type", 0.05, "fleet_trip_enriched")
cre = spark.table("workspace.silver.coldchain_reading_enriched")
assert_max_null_rate(cre, "product_type", 0.30, "coldchain_reading_enriched") # some readings are warehouse-
only, no container
print("Clean joins built and validated: fleet_trip_enriched, coldchain_reading_enriched")
```

WATCH OUT — THE TWO MISMATCHED JOINS

Per the Data Dictionary Part 8, `fleet.vehicle_id` (VEH_000001, 6-digit) cannot be joined directly to driver-domain `vehicle_id` (VEH_0001, 4-digit), and `shipment_data.shipment_id` (SHIP_#####) cannot be joined directly to the delivery domain's `shipment_id` (AGL-#####). Both pairs have matching row counts, which makes a positional crosswalk tempting — build it only as a labeled, reviewable heuristic:

```
# HEURISTIC BRIDGE - validate against a real source system before using this beyond a demo
from pyspark.sql.functions import row_number, col
from pyspark.sql.window import Window

fleet_ordered = spark.table("workspace.silver.dim_vehicle").withColumn(
    "seq", row_number().over(Window.orderBy("vehicle_id")))
)
driver_vehicle_ids = (
    spark.table("workspace.bronze.driver_behavioral_events")
    .select("vehicle_id").distinct()
    .withColumn("seq", row_number().over(Window.orderBy("vehicle_id")))
    .withColumnRenamed("vehicle_id", "driver_domain_vehicle_id")
)

vehicle_id_bridge = (
    fleet_ordered.select("vehicle_id", "seq")
    .join(driver_vehicle_ids, "seq")
    .select("vehicle_id", "driver_domain_vehicle_id")
)

vehicle_id_bridge.write.mode("overwrite").option("overwriteSchema",
"true").saveAsTable("workspace.silver._bridge_vehicle_id_heuristic")
print(f"Bridge built with {vehicle_id_bridge.count()} mappings - review
workspace.silver._bridge_vehicle_id_heuristic manually.")
```

5.11 Pivoting / Unpivoting

```
# PIVOT: milestones (10 rows/shipment) -> one row per shipment, one column per milestone
milestone_wide = (
    spark.table("workspace.bronze.delivery_milestones")
    .groupBy("shipment_id")
```

```

        .pivot("milestone_name", [
            "Booking Confirmed", "Cargo Received", "Departed Origin", "Arrived at Border",
            "Customs Cleared", "Border Cleared", "Departed Border", "Arrived at Destination",
            "Out for Delivery", "Delivered",
        ])
        .agg({"timestamp": "max"})
    )
    assert_not_empty(milestone_wide, "delivery_milestones_wide")
    milestone_wide.write.mode("overwrite").option("overwriteSchema",
"true").saveAsTable("workspace.silver.delivery_milestones_wide")

# UNPIVOT: behavioral_events' 12 sparse, event-type-specific columns -> tall attribute/value table
sparse_cols = [
    "deceleration_rate", "distance_to_obstacle_m", "acceleration_rate", "engine_load_percent",
    "speed_limit_kmh", "excess_speed_kmh", "idle_duration_minutes", "fuel_consumed_l",
    "duration_unbuckled_seconds", "warning_count", "lateral_acceleration", "curve_radius_m",
]
stack_expr = "stack({0}, {1}) as (attribute, value)".format(
    len(sparse_cols), ", ".join([f"'{c}', {c}" for c in sparse_cols])
)
behavioral_long = (
    spark.table("workspace.bronze.driver_behavioral_events")
    .selectExpr("event_id", "driver_id", "event_type", "timestamp", stack_expr)
    .filter("value IS NOT NULL")
)
assert_not_empty(behavioral_long, "driver_behavioral_events_attributes_long")
behavioral_long.write.mode("overwrite").option("overwriteSchema",
"true").saveAsTable("workspace.silver.driver_behavioral_events_attributes_long")

print("Pivot: delivery_milestones_wide. Unpivot: driver_behavioral_events_attributes_long")

```

5.12 Data Masking / Obfuscation

```

from pyspark.sql.functions import sha2, concat, regexp_extract, lit, col

def mask_email(colname):
    return concat(regexp_extract(col(colname), r"^(.)", 1), lit("***@"), regexp_extract(col(colname),
r"@(.*)$", 1))

def mask_phone(colname):
    return concat(lit("***-***-"), col(colname).substr(-4, 4))

dim_driver_masked = (
    spark.table("workspace.silver.dim_driver")
    .withColumn("email_masked", mask_email("email"))
    .withColumn("phone_masked", mask_phone("phone"))
    .withColumn("license_number_hash", sha2(col("license_number"), 256))
    .drop("email", "phone", "license_number", "emergency_contact", "emergency_phone")
)
assert_not_empty(dim_driver_masked, "dim_driver_masked")
dim_driver_masked.write.mode("overwrite").option("overwriteSchema",
"true").saveAsTable("workspace.silver.dim_driver_masked")

# signature blobs carry no analytical value and are PII-adjacent - drop entirely from the analytics-safe copy
pod_analytics_safe = (
    spark.table("workspace.bronze.delivery_proof_of_delivery")
    .drop("recipient_signature", "delivery_agent_signature", "recipient_email", "recipient_contact")
)
assert_not_empty(pod_analytics_safe, "delivery_proof_of_delivery_safe")
pod_analytics_safe.write.mode("overwrite").option("overwriteSchema",
"true").saveAsTable("workspace.silver.delivery_proof_of_delivery_safe")

print("Masked tables ready: dim_driver_masked, delivery_proof_of_delivery_safe")

```

NOTE

Keep the un-masked `silver.dim_driver` tightly permissioned (Unity Catalog grants) and give broad access only to `dim_driver_masked` — one governed source of truth, one safe-to-share derived copy. This is the pattern every Gold table in Part 6 that touches driver data should build from the masked copy, not the raw one.

5.13 Schema Evolution

```
# ingestion-side evolution already on via cloudFiles.schemaEvolutionMode (Part 3.4).
# Silver's full-rebuild writes need the equivalent permission on their own side:
spark.table("workspace.bronze.shipment_data").write.mode("overwrite") \
    .option("overwriteSchema", "true") \
    .saveAsTable("workspace.silver.shipment_data_schema_test")

# for incremental (MERGE-based) tables (Part 5.15) instead of full rebuilds, enable auto-merge for the
session:
spark.conf.set("spark.databricks.delta.schema.autoMerge.enabled", "true")

# clean up the throwaway test table
spark.sql("DROP TABLE IF EXISTS workspace.silver.shipment_data_schema_test")
print("Schema evolution settings confirmed.")
```

`overwriteSchema` is for full `CREATE OR REPLACE` rebuilds (used throughout 5.2–5.12); `autoMerge.enabled` is for the `MERGE INTO` pattern in Part 5.15.

5.14 Partitioning & Liquid Clustering for the highest-volume Silver tables

```
%sql
ALTER TABLE workspace.silver.driver_behavioral_events CLUSTER BY (driver_id, event_type);
ALTER TABLE workspace.silver.coldchain_reading_enriched CLUSTER BY (container_id);
ALTER TABLE workspace.bronze.driver_logs CLUSTER BY (driver_id);

OPTIMIZE workspace.silver.driver_behavioral_events;
OPTIMIZE workspace.silver.coldchain_reading_enriched;
```

These three tables are the largest in the dataset (10,000 / ~4,500 / 22,636 rows respectively). Liquid Clustering keeps rows with the same driver/container co-located on disk so joins and filters on those keys don't have to scan the whole table — worth doing now, cheap at this size, and it means you don't have to remember to re-partition later once the tables are 100x bigger.

5.15 Incremental Silver with `MERGE INTO` (production pattern)

Everything above rebuilds Silver tables from scratch every run — correct and simple at this size, but wasteful once Bronze has millions of rows. `shipment_data` has an `updated_at` column, making it a good candidate for the incremental alternative: process only rows that changed since the last run.

```
from pyspark.sql.functions import col, row_number, desc
from pyspark.sql.window import Window

catalog = "workspace"

spark.sql(f"""
CREATE TABLE IF NOT EXISTS {catalog}.silver._watermarks (
    table_name STRING, last_watermark TIMESTAMP
)""")

def get_silver_watermark(table_name):
    row = spark.sql(f"""
        SELECT last_watermark FROM {catalog}.silver._watermarks WHERE table_name = '{table_name}'
    """).collect()
    return row[0]["last_watermark"] if row else "1900-01-01 00:00:00"

def set_silver_watermark(table_name, new_watermark):
    spark.sql(f"""
```

```

MERGE INTO {catalog}.silver._watermarks t
USING (SELECT '{table_name}' AS table_name, TIMESTAMP('{new_watermark}') AS last_watermark) s
ON t.table_name = s.table_name
WHEN MATCHED THEN UPDATE SET t.last_watermark = s.last_watermark
WHEN NOT MATCHED THEN INSERT (table_name, last_watermark) VALUES (s.table_name, s.last_watermark)
""")

# --- bootstrap step: run the full rebuild (Part 5.2-5.6) ONCE first so silver.shipment_data exists ---
# --- from the second run onward, use this incremental block instead ---

last_wm = get_silver_watermark("shipment_data")
new_bronze_rows = spark.table(f"{catalog}.bronze.shipment_data").filter(col("updated_at") > last_wm)
new_count = new_bronze_rows.count()

if new_count == 0:
    print(f"shipment_data: no changes since {last_wm}")
else:
    w = Window.partitionBy("shipment_id").orderBy(desc("updated_at"))
    deduped = new_bronze_rows.withColumn("rn", row_number().over(w)).filter("rn = 1").drop("rn")
    deduped.createOrReplaceTempView("shipment_changes")

    spark.sql(f"""
        MERGE INTO {catalog}.silver.shipment_data t
        USING shipment_changes s
        ON t.shipment_id = s.shipment_id
        WHEN MATCHED THEN UPDATE SET *
        WHEN NOT MATCHED THEN INSERT *
        """)
    new_wm = new_bronze_rows.agg({"updated_at": "max"}).collect()[0][0]
    set_silver_watermark("shipment_data", new_wm)
    print(f"shipment_data: merged {new_count} changed row(s), watermark advanced to {new_wm}")

```

TIP

Save this as a separate notebook, `02b_silver_incremental_merge`, and swap the Silver/Gold curation job (Part 7.3) to call it instead of the full-rebuild notebook once you're comfortable it produces the same results. Both approaches write to the exact same `workspace.silver.shipment_data` table, so switching is a one-line change in the job definition, not a migration.

PART 6

Gold Layer: A Business-First Design

Notebook: `03_gold_layer`. Every table below is built only from joins verified clean in Part 5.10, and every driver-related table reads from the **masked** Silver copies (Part 5.12) — Gold is what executives and models take at face value, so it never touches raw PII or an unvalidated heuristic bridge.

6.1 The persona-to-table matrix

Persona	Gold object	Business question it answers	Refresh cadence
Finance	<code>gold.fact_revenue_and_penalty_exposure</code>	How much revenue is at risk from SLA penalties, by customer tier and currency?	Every 6h
Operations	<code>gold.vw_fleet_utilization</code>	Which vehicles are underutilized or running long overruns?	Every 6h
Operations	<code>gold.vw_maintenance_cost_leakage</code>	Which vehicles cost the most to maintain per kilometer driven?	Every 6h
Customer Success	<code>gold.vw_shipment_sla_performance</code>	Which customer segments are experiencing late/overdue deliveries?	Every 6h
Compliance / Audit	<code>gold.fact_coldchain_compliance_summary</code>	What % of cold-chain readings stayed within spec, by product type and day?	Every 6h
Data Scientist	<code>gold.fact_coldchain_spoilage_risk</code>	Feature table for a spoilage-risk classifier	Every 6h
Data Scientist	<code>gold.fact_driver_safety_scorecard</code>	Feature table for driver-risk scoring / coaching prioritization	Every 6h
ML Engineer	Feature Store registration of the two tables above	Reusable, point-in-time-correct features for training and serving	On demand

6.2 Finance — `gold.fact_revenue_and_penalty_exposure`

```
spark.sql("""
CREATE OR REPLACE TABLE workspace.gold.fact_revenue_and_penalty_exposure AS
SELECT
    c.customer_value_tier,
    s.currency,
    COUNT(*) AS shipment_count,
    SUM(s.total_cost) AS total_revenue,
    SUM(s.penalty_exposure_amount) AS total_penalty_exposure,
    ROUND(SUM(s.penalty_exposure_amount) / NULLIF(SUM(s.total_cost), 0) * 100, 2) AS
penalty_exposure_pct_of_revenue
FROM workspace.silver.shipment_data s
JOIN workspace.silver.dim_customer c ON s.customer_id = c.customer_id
GROUP BY c.customer_value_tier, s.currency
ORDER BY total_penalty_exposure DESC
""")
print("gold.fact_revenue_and_penalty_exposure ready.")
```

BUSINESS USE

Finance uses this to see, at a glance, which customer tier and currency combination is generating the most penalty risk relative to revenue — a High Value tier with a high `penalty_exposure_pct_of_revenue` is a candidate for

renegotiating SLA terms before the next contract renewal.

6.3 Operations — gold.vw_fleet_utilization

```
spark.sql("""
CREATE OR REPLACE VIEW workspace.gold.vw_fleet_utilization AS
SELECT vehicle_id, vehicle_age_band, truck_type,
       COUNT(*) AS trip_count,
       SUM(actual_distance_km) AS total_km,
       AVG(actual_duration_hours - planned_duration_hours) AS avg_overrun_hours,
       AVG(idle_time_minutes) AS avg_idle_minutes
FROM workspace.silver.fleet_trip_enriched
GROUP BY vehicle_id, vehicle_age_band, truck_type
ORDER BY avg_idle_minutes DESC
""")
print("gold.vw_fleet_utilization ready.")
```

6.4 Operations — gold.vw_maintenance_cost_leakage

```
spark.sql("""
CREATE OR REPLACE VIEW workspace.gold.vw_maintenance_cost_leakage AS
SELECT u.vehicle_id, u.vehicle_age_band, u.total_km,
       m.total_maintenance_cost, m.breakdown_count,
       ROUND(m.total_maintenance_cost / NULLIF(u.total_km, 0), 4) AS maintenance_cost_per_km
FROM workspace.gold.vw_fleet_utilization u
JOIN workspace.silver.fleet_maintenance_cost_by_vehicle m ON u.vehicle_id = m.vehicle_id
ORDER BY maintenance_cost_per_km DESC
""")
print("gold.vw_maintenance_cost_leakage ready.")
```

BUSINESS USE

A vehicle in the Aging (8y+) band with a high maintenance_cost_per_km and a non-zero breakdown_count is a concrete, ranked candidate for fleet replacement — this view is built specifically to sort straight into that decision, not just report a number.

6.5 Customer Success — gold.vw_shipment_sla_performance

```
spark.sql("""
CREATE OR REPLACE VIEW workspace.gold.vw_shipment_sla_performance AS
SELECT c.customer_value_tier, s.delivery_status_flag, s.currency,
       COUNT(*) AS shipment_count,
       SUM(s.total_cost) AS total_revenue,
       AVG(s.penalty_rate) AS avg_penalty_rate
FROM workspace.silver.shipment_data s
JOIN workspace.silver.dim_customer c ON s.customer_id = c.customer_id
GROUP BY c.customer_value_tier, s.delivery_status_flag, s.currency
""")
print("gold.vw_shipment_sla_performance ready.")
```

6.6 Compliance / Audit — gold.fact_coldchain_compliance_summary

```
spark.sql("""
CREATE OR REPLACE TABLE workspace.gold.fact_coldchain_compliance_summary AS
SELECT product_type, CAST(timestamp AS DATE) AS event_date,
       COUNT(*) AS total_readings,
       SUM(temp_excursion_flag) AS excursion_count,
       ROUND((1 - SUM(temp_excursion_flag) / COUNT(*)) * 100, 2) AS compliance_pct
FROM workspace.silver.coldchain_reading_enriched
```

```
WHERE product_type IS NOT NULL
GROUP BY product_type, CAST(timestamp AS DATE)
ORDER BY compliance_pct ASC
""")
print("gold.fact_coldchain_compliance_summary ready.")
```

BUSINESS USE

This is the table a compliance officer pulls for a regulatory audit: "what fraction of Pharmaceuticals readings stayed in spec on any given day" — sorted worst-first so the days needing a closer look surface immediately.

6.7 Data Scientist — gold.fact_coldchain_spoilage_risk

```
spark.sql("""
CREATE OR REPLACE TABLE workspace.gold.fact_coldchain_spoilage_risk AS
SELECT container_id, product_type, CAST(timestamp AS DATE) AS event_date,
       COUNT(*) AS reading_count,
       SUM(temp_excursion_flag) AS temp_excursion_count,
       AVG(temperature_c) AS avg_temp_c, MIN(temperature_c) AS min_temp_c, MAX(temperature_c) AS max_temp_c,
       CASE
         WHEN SUM(temp_excursion_flag) > 5 THEN 'High'
         WHEN SUM(temp_excursion_flag) BETWEEN 1 AND 5 THEN 'Medium'
         ELSE 'Low'
       END AS spoilage_risk_label
FROM workspace.silver.coldchain_reading_enriched
GROUP BY container_id, product_type, CAST(timestamp AS DATE)
""")
print("gold.fact_coldchain_spoilage_risk ready - feature-ready, one row per container-day.")
```

6.8 Data Scientist — gold.fact_driver_safety_scorecard

```
spark.sql("""
CREATE OR REPLACE TABLE workspace.gold.fact_driver_safety_scorecard AS
SELECT d.driver_id, d.experience_band, d.rating, d.rating_rank_in_terminal,
       COUNT(b.event_id) AS behavioral_event_count,
       SUM(CASE WHEN b.severity = 'Critical' THEN 1 ELSE 0 END) AS critical_event_count,
       SUM(CASE WHEN b.event_type = 'Harsh Braking' THEN 1 ELSE 0 END) AS harsh_braking_count
FROM workspace.silver.dim_driver_masked d
LEFT JOIN workspace.silver.driver_behavioral_events b ON d.driver_id = b.driver_id
GROUP BY d.driver_id, d.experience_band, d.rating, d.rating_rank_in_terminal
""")
print("gold.fact_driver_safety_scorecard ready - built from the masked driver dimension.")
```

6.9 ML Engineer — registering Gold tables as Feature Tables

```
from databricks.feature_engineering import FeatureEngineeringClient

fe = FeatureEngineeringClient()
fe.create_table(
    name="workspace.gold.fact_coldchain_spoilage_risk",
    primary_keys=["container_id", "event_date"],
    timestamp_keys=["event_date"],
    df=spark.table("workspace.gold.fact_coldchain_spoilage_risk"),
    description="Container-day cold-chain features for spoilage risk models.",
)

fe.create_table(
    name="workspace.gold.fact_driver_safety_scorecard",
    primary_keys=["driver_id"],
    df=spark.table("workspace.gold.fact_driver_safety_scorecard"),
    description="Per-driver behavioral safety features for risk scoring / coaching prioritization.",
```

```
)  
print("Both feature tables registered in Unity Catalog Feature Engineering.")
```

6.10 Gold-layer reconciliation check (new in v2)

Before trusting any Gold number in a business meeting, confirm it reconciles against Silver — this single query would have caught a bug in the Part 6.2 revenue aggregation before anyone presented it:

```
silver_total = spark.table("workspace.silver.shipment_data").agg({"total_cost": "sum"}).collect()[0][0]  
gold_total = spark.table("workspace.gold.fact_revenue_and_penalty_exposure").agg({"total_revenue":  
"sum"}).collect()[0][0]  
  
diff_pct = abs(silver_total - gold_total) / silver_total * 100  
print(f"Silver total_cost sum: {silver_total:,.2f}")  
print(f"Gold total_revenue sum: {gold_total:,.2f}")  
print(f"Difference: {diff_pct:.4f}%")  
  
if diff_pct > 0.01:  
    raise ValueError(f"Gold revenue reconciliation failed: {diff_pct:.2f}% discrepancy vs Silver - check the  
join/group-by in 6.2.")  
print("Reconciliation passed.")
```

PART 7

CI/CD: Databricks Asset Bundles + GitHub Actions

7.1 Concept: what a Bundle is

CONCEPT

A Databricks Asset Bundle (DAB) is a YAML description (`databricks.yml`) of everything this project needs on Databricks — notebooks, jobs, triggers, parameters — checked into the same Git repo as your code. `databricks bundle deploy` reads it and creates/updates the real jobs in your workspace to match. This turns "click through the Workflows UI 7 times" into a file reviewable in a pull request.

7.2 Repo layout

```
agl-lakehouse/
├─ databricks.yml
├─ notebooks/
│   └─ 00_setup.py
│   └─ 00_data_quality_utils.py
│   └─ 01_bronze_ingestion.py
│   └─ 02_silver_transformation.py
│   └─ 02b_silver_incremental_merge.py
│   └─ 03_gold_layer.py
├─ tests/
│   └─ test_transformations.py
└─ .github/
    └─ workflows/
        └─ deploy.yml
```

7.3 databricks.yml — the full, unabridged config for all 7 jobs

```
bundle:
  name: agl_lakehouse

variables:
  catalog:
    default: workspace
  volume_root:
    default: /Volumes/workspace/landing/agl_volume

targets:
  dev:
    mode: development
    default: true
    workspace:
      host: https://.cloud.databricks.com
  prod:
    mode: production
    workspace:
      host: https://.cloud.databricks.com

resources:
  jobs:
    agl_bronze_shipment:
      name: "AGL Bronze - Shipment"
      tasks:
        - task_key: ingest
```

```

    notebook_task:
      notebook_path: ../notebooks/01_bronze_ingestion.py
      base_parameters: { domain: agl_data_output }
      new_cluster: &small_job_cluster
      spark_version: "15.4.x-scala2.12"
      node_type_id: "Standard_DS3_v2"
      num_workers: 1
  trigger:
    file_arrival:
      url: "${var.volume_root}/agl_data_output/"
  email_notifications:
    on_failure: ["you@example.com"]

agl_bronze_fleet:
  name: "AGL Bronze - Fleet"
  tasks:
    - task_key: ingest
      notebook_task:
        notebook_path: ../notebooks/01_bronze_ingestion.py
        base_parameters: { domain: agl_fleet_data_output }
        new_cluster: *small_job_cluster
  trigger:
    file_arrival:
      url: "${var.volume_root}/agl_fleet_data_output/"
  email_notifications:
    on_failure: ["you@example.com"]

agl_bronze_driver:
  name: "AGL Bronze - Driver"
  tasks:
    - task_key: ingest
      notebook_task:
        notebook_path: ../notebooks/01_bronze_ingestion.py
        base_parameters: { domain: agl_driver_data_output }
        new_cluster: *small_job_cluster
  trigger:
    file_arrival:
      url: "${var.volume_root}/agl_driver_data_output/"
  email_notifications:
    on_failure: ["you@example.com"]

agl_bronze_coldchain:
  name: "AGL Bronze - Coldchain"
  tasks:
    - task_key: ingest
      notebook_task:
        notebook_path: ../notebooks/01_bronze_ingestion.py
        base_parameters: { domain: agl_coldchain_data_output }
        new_cluster: *small_job_cluster
  trigger:
    file_arrival:
      url: "${var.volume_root}/agl_coldchain_data_output/"
  email_notifications:
    on_failure: ["you@example.com"]

agl_bronze_delivery:
  name: "AGL Bronze - Delivery"
  tasks:
    - task_key: ingest
      notebook_task:
        notebook_path: ../notebooks/01_bronze_ingestion.py
        base_parameters: { domain: agl_delivery_data_output }
        new_cluster: *small_job_cluster
  trigger:
    file_arrival:
      url: "${var.volume_root}/agl_delivery_data_output/"
  email_notifications:

```

```

    on_failure: ["you@example.com"]

agl_bronze_external:
  name: "AGL Bronze - External"
  tasks:
    - task_key: ingest
      notebook_task:
        notebook_path: ../notebooks/01_bronze_ingestion.py
        base_parameters: { domain: agl_external_data_output }
        new_cluster: *small_job_cluster
  trigger:
    file_arrival:
      url: "${var.volume_root}/agl_external_data_output/"
  email_notifications:
    on_failure: ["you@example.com"]

agl_silver_gold_curation:
  name: "AGL Silver + Gold Curation"
  schedule:
    quartz_cron_expression: "0 0 */6 * * ?"
    timezone_id: "Africa/Kigali"
  tasks:
    - task_key: silver
      notebook_task: { notebook_path: ../notebooks/02_silver_transformation.py }
      new_cluster: *small_job_cluster
    - task_key: gold
      depends_on: [{ task_key: silver }]
      notebook_task: { notebook_path: ../notebooks/03_gold_layer.py }
      new_cluster: *small_job_cluster
  email_notifications:
    on_failure: ["you@example.com"]

```

7.4 Local validation before pushing

```

pip install databricks-cli
databricks auth login --host https://.cloud.databricks.com

databricks bundle validate -t dev
databricks bundle deploy -t dev
databricks bundle run agl_bronze_fleet -t dev
databricks bundle run agl_silver_gold_curation -t dev

```

7.5 Automated tests (new in v2) — tests/test_transformations.py

A small pytest suite that runs the pure-Python parts of your transformation logic against a local Spark session — no Databricks cluster required, runs in ~30 seconds inside GitHub Actions on every PR.

```

# tests/test_transformations.py
import pytest
from pyspark.sql import SparkSession
from pyspark.sql.functions import regexp_replace, col

@pytest.fixture(scope="module")
def spark():
    return SparkSession.builder.master("local[2]").appName("agl-tests").getOrCreate()

def test_warehouse_mojibake_fix(spark):
    df = spark.createDataFrame([("Ambient (15Â°C to 25Â°C)",)], ["temperature_zone"])
    fixed = df.withColumn("temperature_zone", regexp_replace(col("temperature_zone"), "Â°", "°"))
    result = fixed.collect()[0]["temperature_zone"]
    assert result == "Ambient (15°C to 25°C)"

def test_weight_class_bucketing(spark):

```

```

df = spark.createDataFrame([(500.0,), (3000.0,), (10000.0,), (25000.0,)], ["weight_kg"])
bucketed = df.selectExpr("""
    CASE
        WHEN weight_kg < 1000 THEN 'Light (<1t)'
        WHEN weight_kg < 5000 THEN 'Medium (1-5t)'
        WHEN weight_kg < 20000 THEN 'Heavy (5-20t)'
        ELSE 'Bulk (20t+)'
    END AS weight_class
""")
labels = [r["weight_class"] for r in bucketed.collect()]
assert labels == ["Light (<1t)", "Medium (1-5t)", "Heavy (5-20t)", "Bulk (20t+)"]

def test_email_masking_keeps_first_char_and_domain(spark):
    from pyspark.sql.functions import concat, regexp_extract, lit
    df = spark.createDataFrame([("austingram@example.com",)], ["email"])
    masked = df.withColumn(
        "email_masked",
        concat(regexp_extract(col("email"), r"^(.)", 1), lit("***@"), regexp_extract(col("email"), r"@(.*)$",
1))
    )
    result = masked.collect()[0]["email_masked"]
    assert result == "a***@example.com"

```

```

# requirements-test.txt
pyspark==3.5.1
pytest==8.2.0

```

7.6 GitHub Actions: test, validate, deploy

```

# .github/workflows/deploy.yml
name: Deploy AGL Lakehouse

on:
  pull_request:
    branches: [main]
  push:
    branches: [main, develop]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      with:
        python-version: "3.11"
      - run: pip install -r requirements-test.txt
      - run: pytest tests/ -v

  validate:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: databricks/setup-cli@main
      - name: Validate bundle
        run: databricks bundle validate -t dev
      env:
        DATABRICKS_HOST: ${ secrets.DATABRICKS_HOST_DEV }
        DATABRICKS_TOKEN: ${ secrets.DATABRICKS_TOKEN_DEV }

  deploy_dev:
    if: github.ref == 'refs/heads/develop'
    needs: validate
    runs-on: ubuntu-latest

```

```

steps:
  - uses: actions/checkout@v4
  - uses: databricks/setup-cli@main
  - run: databricks bundle deploy -t dev
  env:
    DATABRICKS_HOST: ${ secrets.DATABRICKS_HOST_DEV }
    DATABRICKS_TOKEN: ${ secrets.DATABRICKS_TOKEN_DEV }
  - run: databricks bundle run agl_bronze_shipment -t dev
  env:
    DATABRICKS_HOST: ${ secrets.DATABRICKS_HOST_DEV }
    DATABRICKS_TOKEN: ${ secrets.DATABRICKS_TOKEN_DEV }

deploy_prod:
  if: github.ref == 'refs/heads/main'
  needs: validate
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: databricks/setup-cli@main
    - run: databricks bundle deploy -t prod
  env:
    DATABRICKS_HOST: ${ secrets.DATABRICKS_HOST_PROD }
    DATABRICKS_TOKEN: ${ secrets.DATABRICKS_TOKEN_PROD }

```

7.7 Wiring up the GitHub ↔ Databricks connection

1. **Create a Databricks Personal Access Token** (or a service principal + OAuth for production) in each workspace: User Settings → Developer → Access tokens → Generate new token.
2. **Store it in GitHub:** repo → Settings → Secrets and variables → Actions → New repository secret. Add `DATABRICKS_HOST_DEV`, `DATABRICKS_TOKEN_DEV`, `DATABRICKS_HOST_PROD`, `DATABRICKS_TOKEN_PROD`.
3. **Push the repo to GitHub**, open a PR from a feature branch into `develop` — the `test` and `validate` jobs run automatically and block the merge on failure.
4. **Merge to `develop`** → `deploy_dev` runs, updates all 7 jobs in dev, and smoke-tests one of them.
5. **Merge `develop` into `main`** → `deploy_prod` runs against production.

TIP

You can also link the repo directly inside Databricks (Workspace → Repos → Add Repo → paste the GitHub URL) for interactive development. That's independent of the CI/CD pipeline above — useful while actively writing notebooks; GitHub Actions is what keeps the workspace's actual jobs in sync with what's merged.

ROLLBACK

Because every deploy is driven by the Git history, rolling back a bad production deploy is `git revert` the merge commit on `main` and letting `deploy_prod` run again — it redeploys the previous job definitions exactly.

PART 8

Troubleshooting Reference

Symptom	Fix
<code>spark.read.format("xml")</code> throws <code>ClassNotFoundException</code>	spark-xml Maven library (Part 1.2) isn't installed on the cluster running the job.
A File Arrival job never fires	Confirm you uploaded into the domain's own subfolder — a job's trigger only watches its configured path.
<code>from_json</code> on <code>last_known_location</code> returns all-null	Skipped the single-quote → double-quote fix in Part 5.2 — the source string isn't valid JSON as-is.
A join in Part 5.10 returns mostly-null enriched columns	Check if it's one of the two confirmed-mismatched keys (<code>vehicle_id</code> 4- vs 6-digit, <code>shipment_id</code> SHIP_ vs AGL-) before assuming a code bug — <code>assert_max_null_rate</code> should already have raised on this.
Part 6.10's reconciliation check raises	A join in an upstream Gold table duplicated rows (fan-out) — check for a one-to-many join where you expected one-to-one before trusting the number.
pytest step fails in CI but passes locally	Confirm <code>requirements-test.txt</code> pins the same PySpark version as your cluster's Databricks Runtime — a version mismatch can change function behavior subtly.
<code>databricks bundle deploy</code> fails with an auth error	Re-check the GitHub secret names match the workflow YAML exactly, and the PAT hasn't expired.
Bronze ingestion re-processes a file already ingested	Should not happen — if it does, confirm the <code>_checkpoints</code> folder wasn't deleted or moved.

PART 9

Glossary

Term	Meaning
Auto Loader / cloudFiles	Databricks' incremental file-ingestion streaming source; tracks processed files in a checkpoint.
File arrival trigger	A Job-level trigger that starts a run within ~1 minute of a new file appearing in a Volume path.
Checkpoint	Hidden folder recording which files/offsets a streaming job has already processed.
Manifest-table fallback	A hand-rolled "already processed" tracking table for formats (like XML) Auto Loader doesn't natively support.
Data quality guardrail	A function (<code>assert_not_empty</code> , <code>assert_max_null_rate</code> , <code>row_count_delta</code>) that raises an exception rather than let a broken transformation silently produce bad output.
Watermark	The timestamp of the most recently processed change, saved so the next incremental run only reads newer rows.
Liquid Clustering	Delta table maintenance mode (<code>CLUSTER BY</code>) that keeps related rows co-located without manual re-partitioning.
Heuristic bridge/crosswalk table	A best-effort key mapping built when two tables' natural keys don't match — must be validated before use in Gold.
Reconciliation check	A query comparing an aggregate between two layers (e.g. Silver vs. Gold revenue totals) to catch transformation bugs before they reach a dashboard.
Databricks Asset Bundle (DAB)	A YAML-defined, version-controlled description of the jobs/notebooks a project needs, deployed via the Databricks CLI.
Feature Engineering Client	The Unity Catalog-backed API for registering Gold tables as reusable, point-in-time-correct ML feature tables.

AGL Lakehouse — Medallion Implementation & CI/CD Guide v2. Built directly on the verified findings in [AGL_Data_Dictionary_Verified.pdf](#).